

SPRING BOOT

```
package com.example.springweb.controller;

import java.util.List;
import java.util.Optional;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.springweb.model.StudentsModel;
import com.example.springweb.service.StudentsService;

@RestController
@RequestMapping("/jpa")
public class StudentsController {

    private StudentsService service;

    // DI
    public StudentsController(StudentsService service){
        this.service = service;
    }

    @GetMapping("/list")
    public List<StudentsModel> getStudent(){
        return service.getAllstudent();
    }

    @GetMapping("/list/{id}")
    public ResponseEntity<StudentsModel> getStudentById(@PathVariable int id){
        return ResponseEntity.status(200).body(service.getStudentsId(id));
    }

    @PostMapping("/addStudent")
    public ResponseEntity<StudentsModel> saveSStudent(@RequestBody
StudentsModel student){
        System.out.println(student.getName());
        System.out.println(student.getAge());
        System.out.println(student.getCourse());
        return ResponseEntity.status(201).body(service.saveStudents(student));
    }
}
```

```
}
```

```
package com.example.springweb.exception;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>>
handleNotFound(ResourceNotFoundException ex){

        Map<String, Object> error = new HashMap<>();

        error.put("timestamp", LocalDateTime.now());
        error.put("message", ex.getMessage());
        error.put("status", 404);

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

}
```

```
package com.example.springweb.exception;

public class ResourceNotFoundException extends RuntimeException{

    public ResourceNotFoundException(String message){
        super(message); // calls parent constructor
    }

}
```

```
package com.example.springweb.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "student")
public class StudentsModel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String name;
    private int age;
    private String course;
    public StudentsModel(Integer id, String name, int age, String course) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.course = course;
    }
    public StudentsModel() {
    }
    public Integer getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public String getCourse() {
        return course;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public void setCourse(String course) {
```

```
        this.course = course;
    }

}
```

```
package com.example.springweb.Repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.springweb.model.StudentsModel;

public interface StudentsRepository extends JpaRepository<StudentsModel,
Integer>{

}
```

```
package com.example.springweb.service;

import java.util.List;
import java.util.Optional;

import org.springframework.stereotype.Service;

import com.example.springweb.Repository.StudentsRepository;
import com.example.springweb.exception.ResourceNotFoundException;
import com.example.springweb.model.StudentsModel;

@Service
public class StudentsService {

    private StudentsRepository repo;

    public StudentsService(StudentsRepository repo){
        this.repo = repo;
    }

    public List<StudentsModel> getAllstudent(){
        return repo.findAll();
    }

    public StudentsModel getStudentsId(int id){
        return repo.findById(id).orElseThrow( ()->new
ResourceNotFoundException("Student not found") );
    }
}
```

```
}

public StudentsModel saveStudents(StudentsModel student){
    return repo.save(student);
}

}
```

Theory

- **Controller Layer** → Handles HTTP requests (@RestController). Uses ResponseEntity to return status codes and responses.
- **Service Layer** → Encapsulates business logic. Calls repository methods.
- **Repository Layer** → Extends JpaRepository to provide CRUD operations.
- **Model Layer** → Annotated with @Entity and @Table to map to database tables.
- **Exception Handling** →
 - @ControllerAdvice → Global exception handler.
 - @ExceptionHandler → Handles specific exceptions.
 - Custom exceptions like ResourceNotFoundException provide meaningful error messages.
- **ResponseEntity** → Used to send HTTP status codes (200 OK, 201 Created, 404 Not Found) along with data.



Coding Round Questions

- Get all students → Write a controller method that fetches all students from the database and returns them as JSON.
- Get student by ID → Implement a method that fetches a student by ID. If not found, throw ResourceNotFoundException.
- Add student → Create a POST endpoint to add a new student to the database and return 201 Created.
- Update student → Implement a PUT endpoint to update student details by ID.
- Delete student → Write a DELETE endpoint to remove a student by ID. If not found, return 404.
- Global exception handling → Implement a global exception handler that returns a JSON error response with timestamp, message, and status code.

Machine Coding Round Challenge

Problem: Complete Student Management API with Exception Handling

Requirements:

1. Create a StudentsModel entity with fields: id, name, age, course.
2. Implement CRUD endpoints:
 - GET /jpa/list → fetch all students.
 - GET /jpa/list/{id} → fetch student by ID.
 - POST /jpa/addStudent → add new student.
 - PUT /jpa/update/{id} → update student details.
 - DELETE /jpa/delete/{id} → delete student.
3. Use ResponseEntity to return proper HTTP status codes.
4. Throw ResourceNotFoundException if student not found.
5. Implement GlobalExceptionHandler with @ControllerAdvice to handle exceptions and return JSON error responses.